

Kilosort4 spike sorting SOP

个人整理，仅供参考，如果错误，欢迎指正

目录

1	背景简介	3
1.1	Kilosort4 的原理	3
1.1.1	第一步：预处理（信号清洗 + 内存优化）	3
1.1.2	第二步：模板去卷积（Spike 检测 + 特征提取）	3
1.1.3	第三步：图聚类（自适应分类）	4
1.1.4	漂移校正（迭代收敛）	4
1.2	传统 sortig 原理	5
2	标准化操作清单	6
3	软硬件环境配置	6
3.1	Windows 系统硬件要求	6
3.2	Python、CUDA 与 PyTorch	7
3.3	GUI 安装与启动	7
4	数据加载机制	8
4.1	SpikeInterface 转换流程	8
4.2	第三方记录系统格式示例	9
5	Probe 文件定制	9
6	参数系统详解	11
7	调参原则与重点参数	13
7.1	调参原则	13
7.2	重点参数归纳	14
8	输出文件解析	17
9	参考资料	19

1 背景简介

Kilosort 是面向大规模电生理记录的 spike sorting 工具包，最新版本是 Kilosort4，详见 Pachitariu 等人 2024 年的 Nature Methods 论文。Kilosort4 的核心定位是：通过在 GPU 上进行预处理、漂移校正、基于模板的检测与聚类，把成百上千个通道记录的混合电压信号，拆解成单个神经元的放电时间序列； Kilosort4 可将结果输出为 Phy 的直接读取格式，将自动分选结果交给 Phy 进行人工复核。

Algorithms	Language	Preprocessing		Template deconvolution				Clustering and postprocessing		
		Filtering and whitening	Drift correction	Template deconvolution	Template learning	Deconvolution during learning	New templates from residual	Clustering	Splits	Merges
Kilosort1 (2016)	MATLAB + CUDA	Yes [§]	–	Yes [§]	Scaled k-means [§]	–	–	–	–	–
Kilosort2 (2018)	MATLAB + CUDA	Yes	– (only tracking [§])	Yes	Scaled k-means	Yes [§]	Threshold crossing [§]	–	Bimodality pursuit [§]	Yes [§]
Kilosort2.5 (2020)	MATLAB/Python + CUDA	Yes	Yes [§]	Yes	Scaled k-means	Yes	Threshold crossing	–	Bimodality pursuit	Yes
Kilosort3 (2021)	MATLAB + CUDA	Yes	Yes	Yes	Recursive pursuit [§]	–	–	Recursive pursuit [§]	Yes	Yes
Kilosort4 (2023)	Python + pytorch	Yes	Yes	Yes	Graph clustering [§]	–	–	Graph clustering [§]	Merging tree [§]	Yes

Bold text indicates new features added after Kilosort1, in the version where they were first introduced. [§]Described in this paper.

表 1: Kilosort 的发展历程

1.1 Kilosort4 的原理

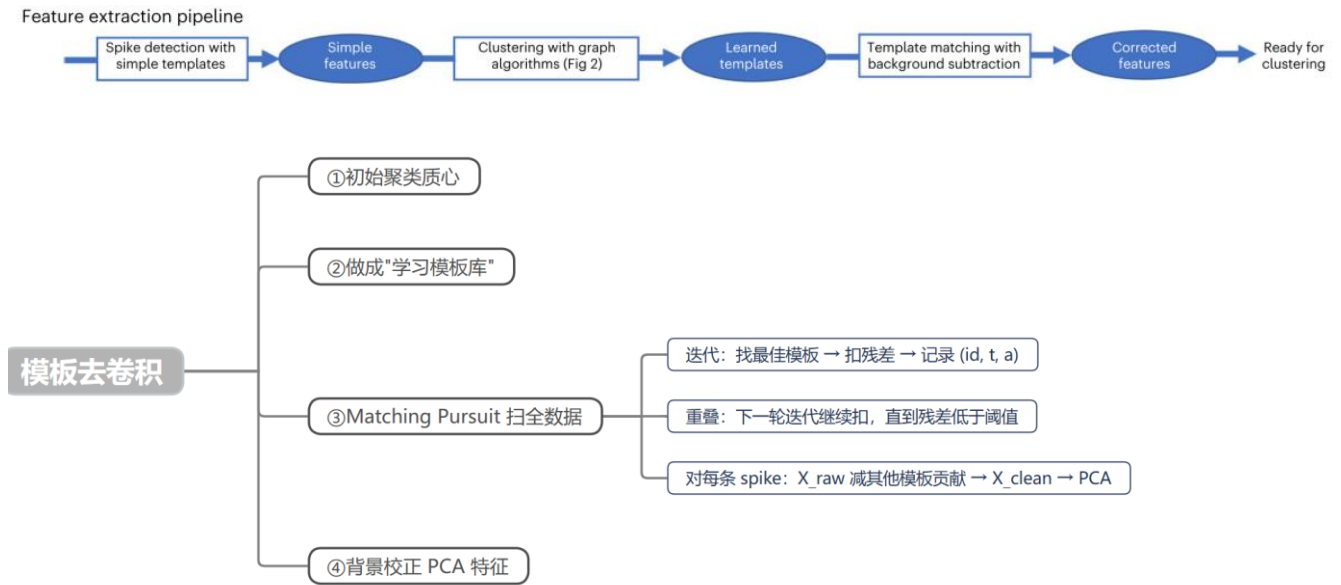
1.1.1 第一步：预处理（信号清洗 + 内存优化）

- 1) KS4 改用 Python + PyTorch，默认分批(batch)加载而非一次性读入，内存占用大幅压低。
- 2) 公共平均参考(CAR)：每个通道减去邻近通道中位数，削掉工频、光遗传伪迹等远场噪声。
- 3) 300Hz 高通 FIR 滤波：滤低频漂移，FFT 卷积加速。
- 4) ZCA 白化：对 32 邻域通道的协方差矩阵求逆，消除通道间空间相关性，让噪声变成各向同性高斯，白化后波形幅值会变，但"形状差异"被放大，聚类更稳，这一步直接决定后面聚类的可分性。

1.1.2 第二步：模板去卷积（Spike 检测 + 特征提取）

- 1) 这是 KS 系列的灵魂，KS4 的关键变化是"模板 ≠ 最终单元"，检测与分类解耦了。
- 2) 两遍扫描：第一遍利用通用模板初检：用预设的一组空间高斯模板扫数据，过阈值抓负峰，做单通道 PCA，K-means 得到初始波形库；第二遍用学习模板重扫：用第一轮的真实神经元模板再扫一次，更准地抓低幅 spike、对齐时间戳。
- 3) 匹配追踪：模板库逐个匹配原始数据，找到最佳匹配就把它从数据里减掉（包括重叠 spike 的残差扣除），重复迭代。

- 4) 产出背景校正 PCA 特征；扣除"邻近神经元 + 噪声背景"后的纯净 PCA 投影，作为下一步聚类的输入，即先清洗特征，再聚类。



Spike 检测与特征提取流程图

1.1.3 第三步：图聚类（自适应分类）

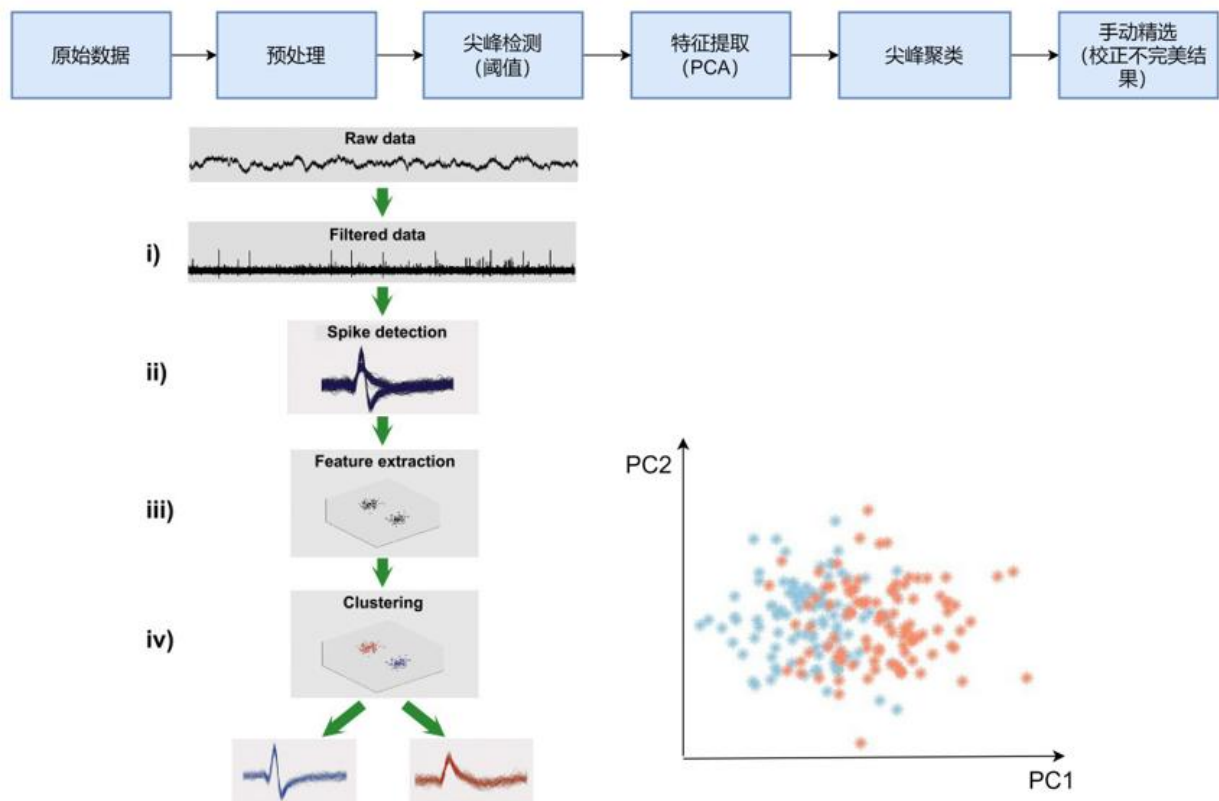
- 1) 构建邻接图：在探针局部区段（~40 μm 垂直窗）内，按 PCA 特征 + 空间位置算 spike 间欧氏距离。
- 2) Louvain / Leiden 粗分：最大化图模块度自动划簇，不用预设簇数，适应局部密度变化。
- 3) 分裂 / 合并树细分：初划分很糙，需要进一步修正；当某个社区内部特征呈双峰分布，且交叉相关图在不应期内出现反常堆积（违反神经元生理规律）时，判定为混入了多个神经元，执行拆分；当两组单元的波形高度相似（相关系数 > 0.5 ），且交叉相关图呈现零延迟凹陷（符合单一神经元的折射特性）时，判定为同一神经元被误拆，执行合并。

1.1.4 漂移校正（迭代收敛）

- 1) 底层逻辑：白化后、KS4 先用预设通用模板 + 宽松阈值做一次“临时粗糙检测”，拿到一批 proxy spike；基于它们的深度分布直方图估计非刚性位移场；真正的高精度 spike 是在漂移校正启用后的正式 detect_spikes 里才产生的。
- 2) 非刚性漂移跟踪：Kilosort4 先通过粗检测锋电位构建时空振幅直方图，迭代配准算出随时间变化的垂直漂移曲线，并生成非刚性位移场插值校正；sorting 时依据漂移量用高斯核对原始电压做空间重采样平移对齐，同步融合白化线性变换加速运算，将不同时刻电极相对神经元的位移归一至统一基准空间，消除漂移导致的波形偏移与神经元分裂问题。“非刚性”的意思是：探针不同深度段漂移幅度可能不一样，不能一刀切刚性平移。

3) 残差重挖掘：从"已减模板的残差"里再阈值扫一遍，抓漏检的低幅 spike，生成新模板补进库，经过 2-3 轮迭代模：板匹配 → 去卷积 → 聚类 → 漂移校正循环跑，逐渐收敛。

1.2 传统 sortig 原理



Offline Sorter (Plexon) 是经典的锋电位分类工具，核心原理是：对在线/离线滤波后的多通道电生理信号，先逐通道或跨通道设定幅值阈值检测过阈的尖峰（spike），截取固定时间窗波形片段，经过 PCA 特征提取后，再通过手动或半自动的方式在特征空间中进行 K-means/手动圈选聚类，结合波形形状、通道位置、ISI（不应期）交叉相关图人工校验合并/分裂，最终将混合信号划分为单神经元放电序列，依赖强人工交互，对高密度重叠 spike 和漂移处理能力弱，适合低密度电极（如钨丝阵列等传统电极）

与 Offline Sorter 相比，Kilosort4 的优势主要体现在高通量、批处理、GPU 加速和标准化输出。官方文档明确指出，CPU 模式运行只适合测试，速度远慢于 GPU；官方文档也建议从默认参数开始，再根据 Phy 中看到的结果适当反向调整参数。其局限是：对硬件环境、probe layout、二进制数据储存格式和参数一致性较敏感；低通道数或稀疏探针可能需要谨慎调参，甚至关闭漂移校正。

维度	Kilosort4 自动化流程	Offline Sorter 工作流
运行方式	GPU/PyTorch 加速，批处理长时间序列；输出可直接进入 Phy。	依赖传统人工阈值、聚类与可视化复核；吞吐量通常较低。

核心对象	模板、通道邻域、漂移校正、PC 特征、自动 good/mua 标记。	阈值事件、波形特征、人工聚类边界与单元审核。
优势	适合 Neuropixels 等高通道数据；流程可复现，文件结构标准。	交互直观，便于小规模数据和人工判断。
风险	probe 坐标错误、CUDA/PyTorch 安装错误会直接影响结果。	人工操作差异大，批量数据处理效率较低。

2 标准化操作清单

阶段	检查项	通过标准
①安装	Windows 10/11、conda 环境、kilosort[gui]、GPU PyTorch。	python -m kilosort 可启动 GUI，PyTorch 可识别 CUDA GPU。
②数据	Binary, row-major/C order、dtype、fs、n_chan_bin。 时间1: ch0 ch1 ch2 ... chN 时间2: ch0 ch1 ch2 ... chN 时间3: ch0 ch1 ch2 ... chN	GUI load 后热图时间轴与通道布局合理，无重复斜线。
③电极布局	chanMap、xc、yc、kcoords、n_chan。	Preview Probe 布局与真实探针一致，n_chan 与实际通道一致。
④调参运行	默认参数起步，仅按现象小步调整。	Run 完成并生成 kilosort4 结果目录。
⑤校正	Phy 打开 params.py，检查 good/mua、ACG/CCG、波形稳定性等。	必要时回到参数页按建议调阈值、漂移或布局参数等。

3 软硬件环境配置

3.1 Windows 系统硬件要求

Kilosort4 支持 Windows10/11 64-bit，GPU 显存至少 8 GB，推荐 12 GB 或更多。官方文档以 384 通道、少于 3 小时的 Neuropixels 探针记录为标准场景，推荐 NVIDIA GPU、SSD、本机 RAM 至少 32 GB，以及 8 核 以上 CPU。

项目	官方建议	操作要点
操作系统	Windows 10/11 已完整测试。	Windows 下建议使用 Anaconda Prompt 或具备 conda 的命令行。
显卡	NVIDIA GPU；至少 8 GB 显存，推荐 12 GB 以上。	确保 NVIDIA 驱动安装正确，并安装匹配 CUDA 的 PyTorch。

存储	推荐 SSD；HDD 会显著拖慢 sorting。	将格式正确的二进制数据复制到本地 SSD 后运行。
内存&CPU	一般 32 GB RAM；8 核以上 CPU；长记录可能需要更多 RAM。	长时间或高 spike 数据需预留更多内存。

3.2 Python、CUDA 与 PyTorch

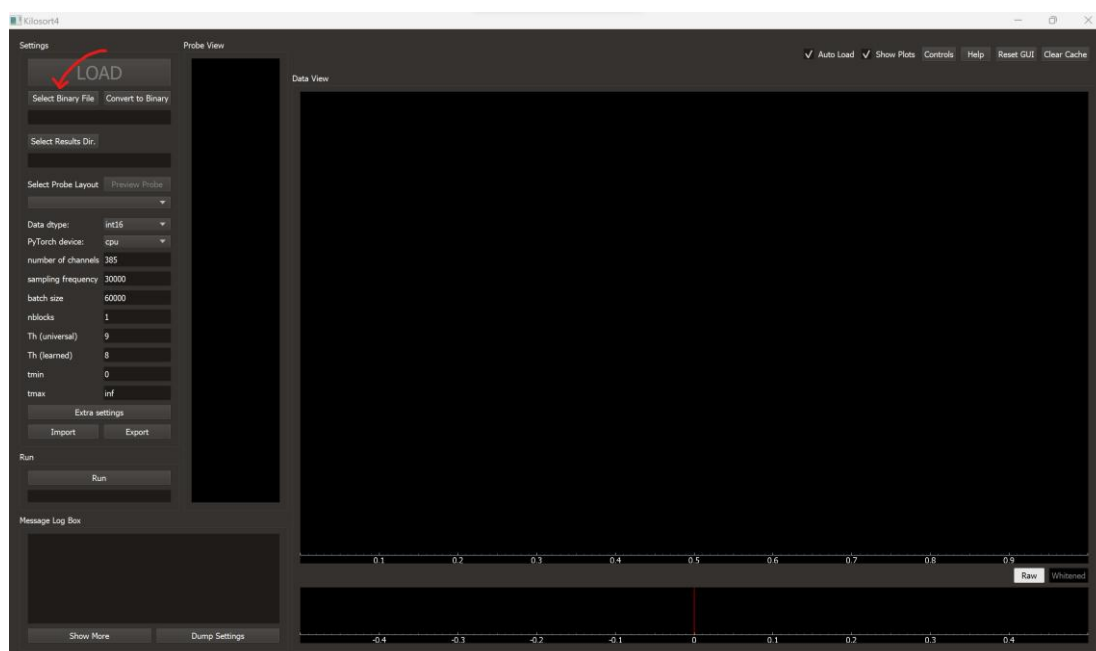
官方安装推荐 Anaconda/conda，创建环境时可使用 Python 3.11，或任意 Python ≥ 3.9 。安装 GUI 版本后，如需 GPU 运行，官方建议卸载 CPU 版 torch，并安装 CUDA 对应的 PyTorch wheel。CUDA 11.8 是其测试与开发的推荐版本之一；若显卡支持的 CUDA 版本不同，应按 PyTorch 官方安装页选择合适命令。如较新的 GPU 如 GeForce RTX 5000 系列，配合 CUDA 12.8 最佳。

```
conda create --name kilosort python=3.11
conda activate kilosort
python -m pip install kilosort[gui]
pip uninstall torch
pip3 install torch --index-url https://download.pytorch.org/whl/cu118
```

3.3 GUI 安装与启动

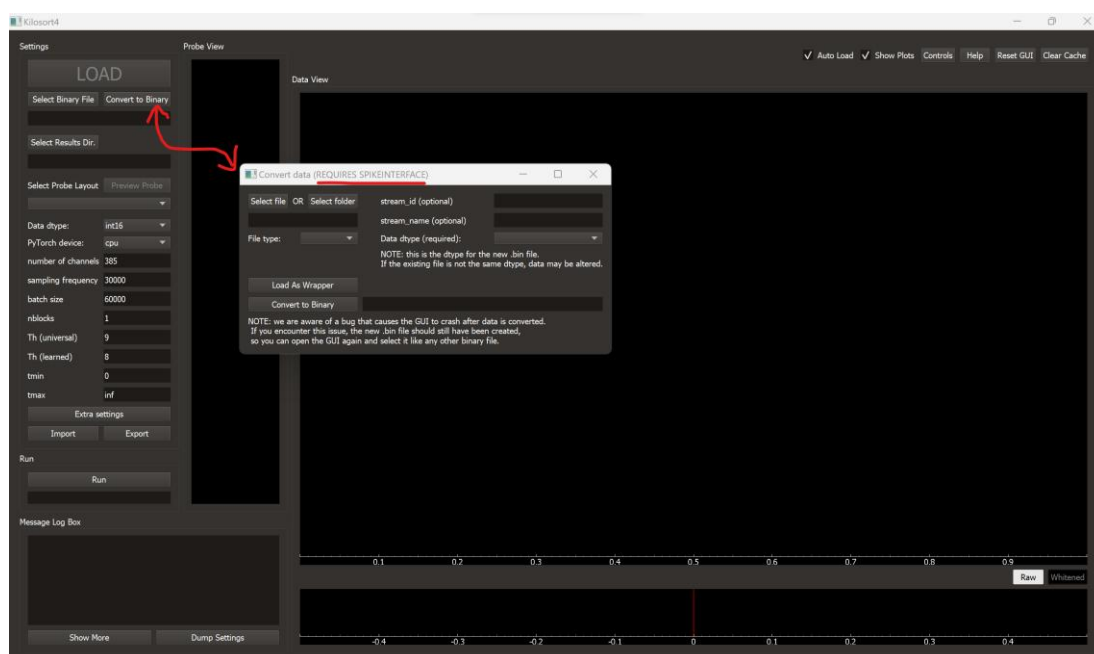
启动 GUI 的命令为 `python -m kilosort`；GUI 操作具体流程包括：选择 .bin/.bat/.dat/.raw 二进制数据；指定结果目录；创建/选择 probe layout；确认 number of channels、dtype、sampling frequency 和计算配置；点击 Load 查看数据；确认数据无明显布局错误后再进行下一步操作。

```
conda activate kilosort
python -m kilosort
```



4 数据加载机制

Kilosort4 原生支持二进制 .bin 数据。如果你收集的数据不是这种格式的，可用 SpikeInterface 将常见电生理格式统一读成 RecordingExtractor，再转换为 Kilosort4 可读取的 binary 文件。对于已是二进制格式的数据，Kilosort4 可直接读取，如 Neuropixels/SpikeGLX 数据通常以 binary 格式保存，不必通过 SpikeInterface 加载，直接将 Kilosort4 指向 .bin 文件即可。需要注意的是 n_chan_bin 必须等于 binary 文件总通道数，包括不参与 ephys sorting 的通道；例如 Neuropixels 1.0 常见为 385 个 binary 通道，其中 384 个为 ephys trace，剩余一个为同步时钟通道（SYNC），而非脑电数据。



GUI 中的 Convert to Binary 工具

4.1 SpikeInterface 转换流程

首先安装 `spikeinterface[full]`，使用 `read_<format>` 读取原始文件，如果 `RecordingExtractor` 中有 probe 信息，可同时导出 .prb；最后通过 GUI 或 API 运行 Kilosort4。下面以读取 Blackrock 记录系统的 .nsX 文件为例：

```
pip install spikeinterface[full]
```

```
# Loading other data formats with SpikeInterface -- kilosort4
from pathlib import Path
import numpy as np
import os
from spikeinterface.extractors import read_blackrock
```



```
# Specify the path where the data will be copied to, and where Kilosort 4
# results will be saved.
DATA_DIRECTORY = Path('C:/Users/ZYH/Desktop') # NOTE: You should change this
# Create path if it doesn't exist
DATA_DIRECTORY.mkdir(parents=True, exist_ok=True)

# Specify path to your existing data
filepath = Path("C:/Users/ZYH/Desktop/file_spec_3_0.ns6") # NOTE: You must change this
# Load existing data with spikeinterface
recording = read_blackrock(filepath)
```

4.2 第三方记录系统格式示例

格式	官方示例读取函数	注意事项
NWB	read_nwb_recording(filepath)	可能需要 electrical_series_name 等额外参数，报错会提示必填项。
SpikeGLX	read_spikeglx(filepath)	一般无需这样加载，直接使用 .bin 即可。
Blackrock	read_blackrock(filepath)	提供 nsX 文件路径，不是 nev 文件；示例为 .ns6。
Neuralynx	read_neuralynx(filepath)	提供包含 .Ncs 文件的目录。
Open Ephys	read_openephys(filepath)	多 stream 数据需指定 stream_id。
Intan	read_intan(filepath, stream_id='0')	若不指定 stream_id，错误信息会解释各个 stream。 Multiple streams found in file. Available streams: '0': RHD2000 amplifier data (64 channels, 30000 Hz) '1': digital input (16 channels, 30000 Hz) '2': auxiliary input (8 channels, 30000 Hz) Please specify stream_id.

此外，Kilosort4 支持 Load As Wrapper 模式，允许通过 API 直接读取 SpikeInterface Recording 对象，无需预先将原始数据转换为 Binary 文件。然而，官方文档明确指出，Copy-to-Binary（预复制为二进制文件）仍是多数场景下的首选方案。这是因为 Binary 格式是通用的中间标准，既能被 Kilosort4 GUI 直接加载，也是下游分析工具 Phy 的唯一直接兼容格式；相比之下，Wrapper 模式生成的临时数据流无法直接对接 Phy 的可视化与手动筛选流程。

5 Probe 文件定制

Kilosort4 使用 probe dictionary 来记录探针布局信息。创建电极布局文件要求字典包含 chanMap、xc、yc、kcoords 和 n_chan，且均对应 NumPy ndarray 或通道数值。坐标单位为微米，且只有描述实际参与记录的 contact centers。

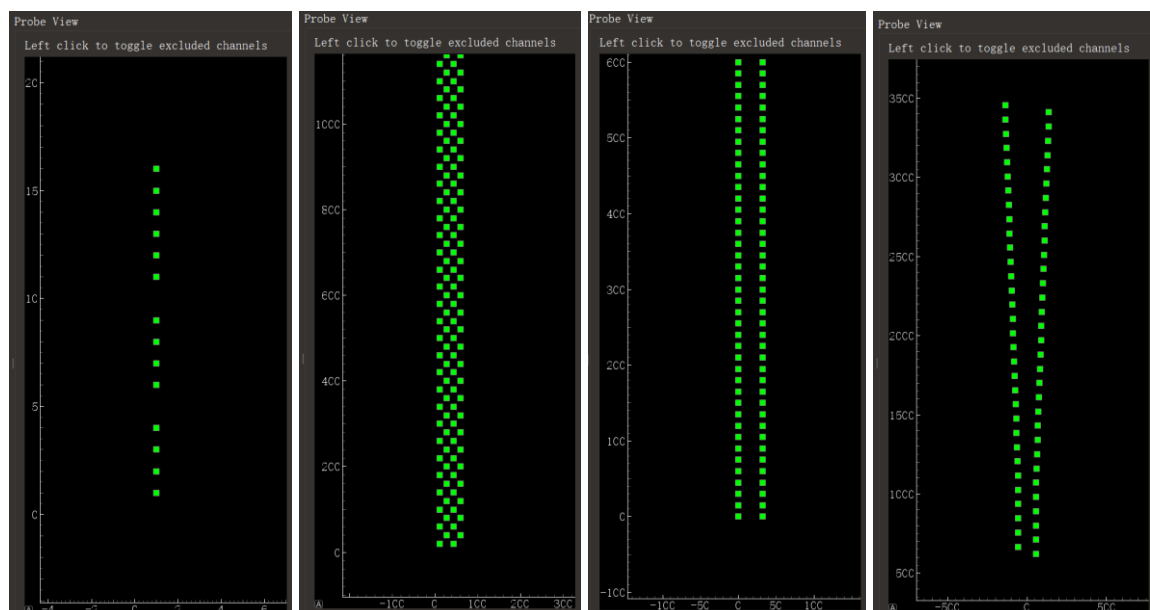
字段	含义	定义规则
chanMap	binary 文件中纳入 ephys 数据的通道索引。	整数索引，映射到 binary 文件的行/通道；全部 128 通道可用 np.arange(128)。
xc	各 contact center 的 x 坐标。	单位 micrometers；长度等于 实际 ephys 通道数。
yc	各 contact center 的 y 坐标。	单位 micrometers；用于深度、漂移和模板空间布局。
kcoords	每个 contact 所属的 shank 或 channel group。	同一 shank 使用相同值；多 shank 使用不同值。
n_chan	probe 中 ephys 通道数量。	不一定等于 n_chan_bin；后者是 binary 文件总通道数。

以美国 Diagnostic Biochips (DBC) 的硅探针 P128-1 为例：

<https://20374082.fs1.hubspotusercontent-na1.net/hubfs/20374082/Product%20PDFs/P128-1.pdf>

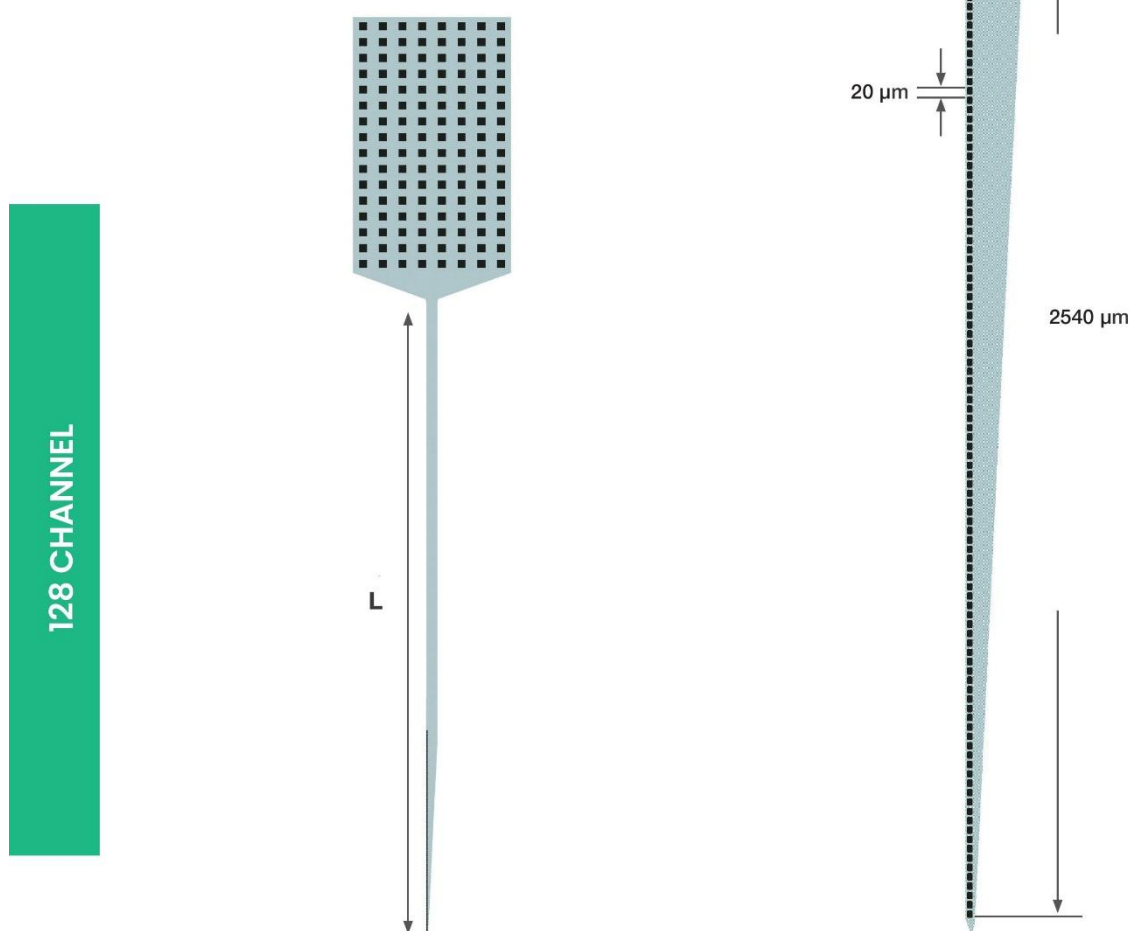
```
import numpy as np
chanMap = np.arange(128)
kcoords = np.zeros(128)
n_chan = 128
xc = np.ones(128) * 5.5
yc = np.array([7.5 + 20*i for i in range(128)])
probe = {'chanMap': chanMap, 'xc': xc, 'yc': yc, 'kcoords': kcoords, 'n_chan': n_chan}
```

在 GUI 中可选择已有 .mat、.prb 或 .json probe；若选择新建，x/y 坐标可用 NumPy 表达式输入，各字段除名称外必须与 ephys 通道数量一致，并可通过 Preview Probe 检查布局是否正确。



Kilosort4 GUI：电极布局预览图

SPECIFICATION	DIMENSIONS
Shank length (L)	8 mm
Shank thickness	15 μm
Recording site	11 μm $\times$ 15 μm



硅探针 P128-1

6 参数系统详解

Kilosort4 参数分主要参数和额外参数，额外参数又为 data、preprocessing、spike detection、clustering、postprocessing 等五步。不要直接复用 Kilosort 旧版本的参数，应从 Kilosort4 默认参

数开始，再根据 Phy 中的结果调整。在 Kilosort4 GUI 界面中，鼠标悬停（hover）在参数名或参数输入框上方时，默认会弹出一个提示信息，内容是该参数的文字说明，通常包括参数的含义、作用、单位及取值建议。下表整理了官方 parameters.py 中核心参数的默认值与含义：

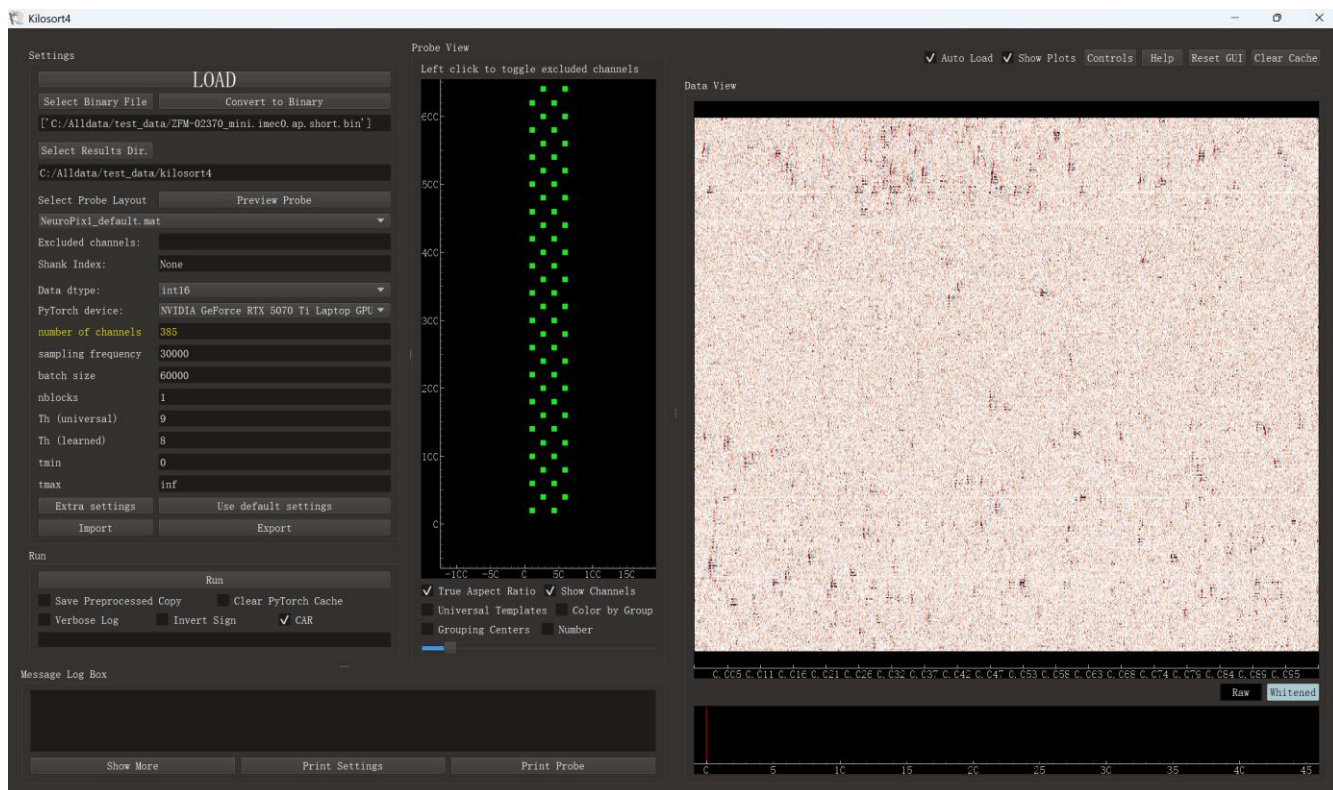
参数	默认值	阶段	物理/算法含义
n_chan_bin	None，API 必填	main parameters	binary 文件总通道数，包括非 ephys 通道；设置错误会导致数据热图异常。
fs	30000	main parameters	采样频率，单位 Hz。
batch_size	60000	main parameters	每批样本数；30 kHz 下对应约 2 s。
nblocks	1	main parameters	漂移校正分块数；1 为刚性配准，0 表示跳过漂移校正。
Th_universal	9	main parameters	universal templates 检测阈值，类似旧版 Th(1)。
Th_learned	8	main parameters	learned templates 检测阈值，类似旧版 Th(2)。
tmin/tmax	0 / inf	main parameters	用于 sorting 的起止时间，单位秒。
nt	61	data	波形样本数与滤波对称 padding；必须为奇数，30 kHz 下约 2 ms + 1 bin。
artifact_threshold	inf	preprocessing	批次绝对值超过阈值时置零；默认不触发。
nskip	25	preprocessing	计算 whitening matrix 的 batch stride。
whitening_range	32	preprocessing	估计 whitening matrix 使用的邻近通道数。
highpass_cutoff	300	preprocessing	高通 Butterworth 滤波临界频率。
dmin	None	spike detection	模板中心垂直间距，默认自动根据 contact 距离估计。
dminx	32	spike detection	模板中心水平间距，Neuropixels 1/2 通常适用。
min_template_size	10	spike detection	universal templates 最小高斯空间包络标准差，单位微米。
template_sizes	5	spike detection	universal templates 空间尺度数量。
nearest_chans	10	spike detection	检测局部极大值时考虑的最近通道数。
nearest_templates	100	spike detection	检测局部极大值时考虑的最近模板位置数。
max_peels	100	spike detection	matching pursuit 每批迭代次数；更多迭代可能检测更多重叠 spikes。

n_templates	6	spike detection	从数据估计 universal templates 时的单通道模板数。
n_pcs	6	spike detection	提取 spike features 使用的单通道 PC 数。
Th_single_ch	6	spike detection	计算 universal templates 的单通道阈值，单位为 whitened data 标准差。
acg_threshold	0.2	clustering	good units 判定中 ACG 不应期违例容忍比例。
ccg_threshold	0.25	clustering	split/merge 中 CCG 不应期违例容忍比例。
cluster_neighbors	10	clustering	构建聚类邻接图时搜索的最近 spike 邻居数。
cluster_downsampling	20	clustering	聚类 landmarks 的 spike 使用比例倒数；与内存和速度相关。
max_cluster_subset	25000	clustering	构建聚类邻居图时每中心最多使用 spike 数。
x_centers	None	clustering	模板分组中心的 x 位置数量；2D array 探针可手动指定。
duplicate_spike_ms	0.25	postprocessing	同一 cluster 中短间隔重复 spike 的移除窗口，单位 ms。
position_limit	100	postprocessing	估计 spike position 时可用通道最大距离，单位微米。

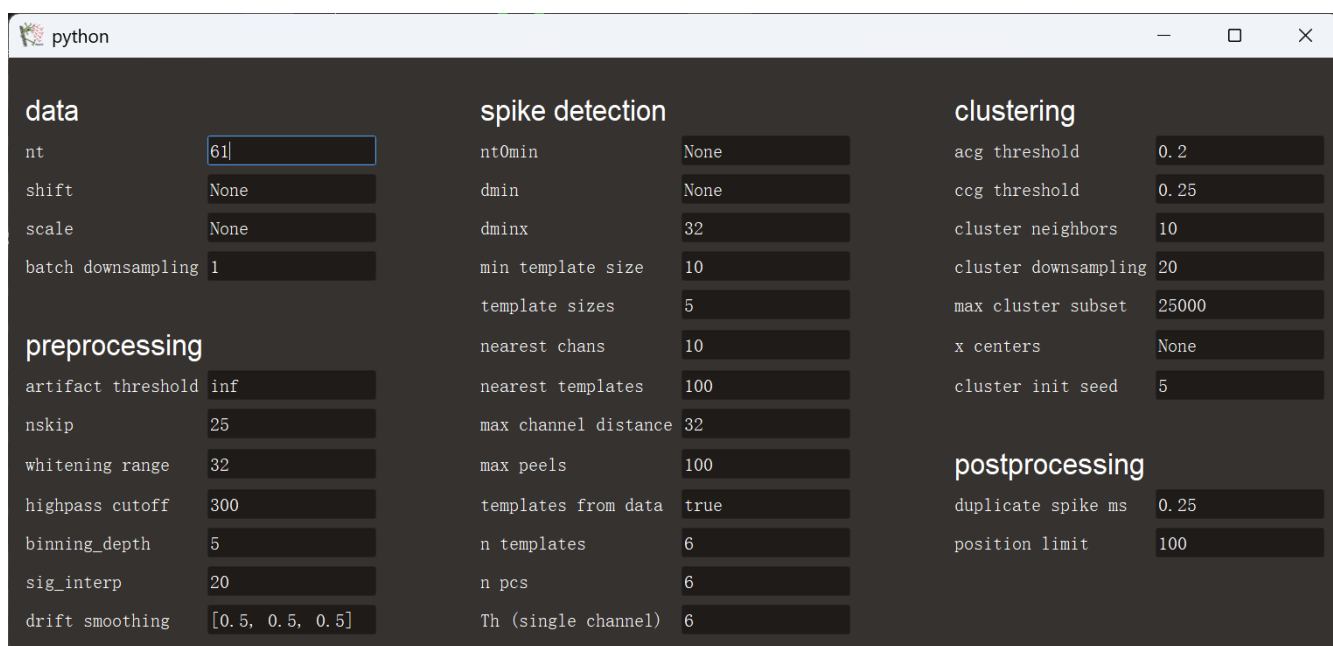
7 调参原则与重点参数

7.1 调参原则

参数调优应遵循渐进式原则，避免一次性大幅改动多个参数，建议由果索因，先观察分类结果的好坏，再针对性地回溯并微调相关参数，以实现精准优化。标准做法是：除了必要的参数需要进行修改外，大部分先用默认参数运行，检查 GUI 中预处理热图、Probe 预览、Phy 中单元稳定性、correlogram 和 good/mua 标记，再小步修改关键参数。



GUI 面板及主要参数设置区域



额外参数设置区域

7.2 重点参数归纳

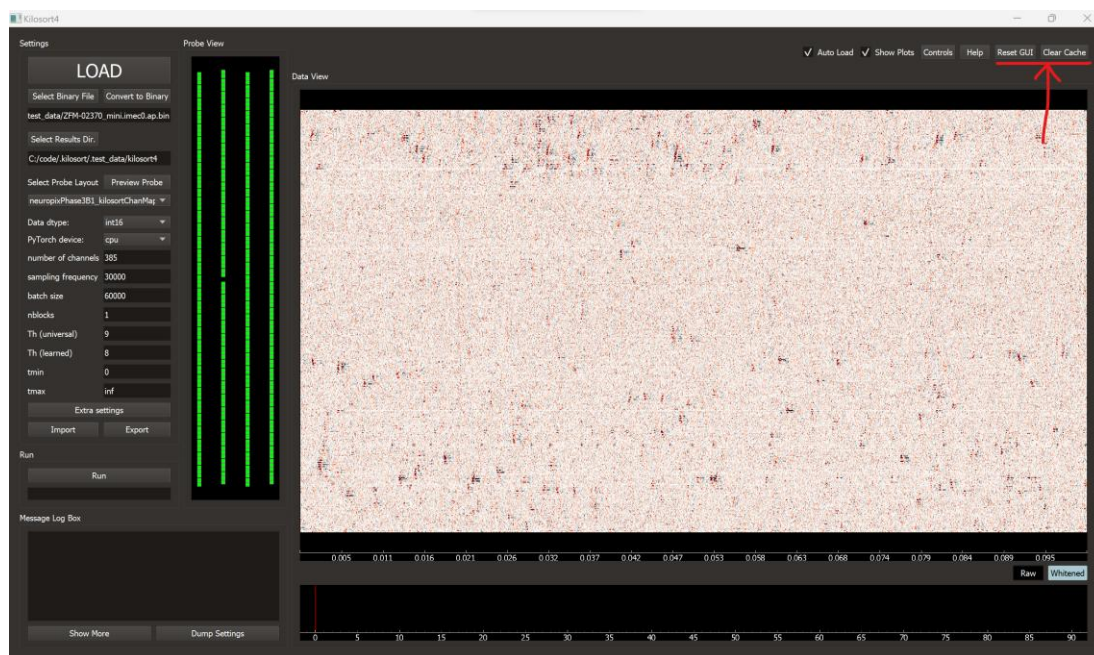
默认参数并非需要全部修改，下面将说明在何种情况下需要改变特定参数以及调整原因。除下表列出的重点参数外，所有可配置参数的简短说明均可在官方 [kilosort/parameters.py](#) 中查看。表中“调整方向”不是固定处方，而是用于定位问题的起点；每次调整后均应回到 GUI 热图、Phy 波形、ACG/CCG 与时间稳定性等结果中进行复核。

参数	适用现象	调整原则与注意事项
n_chan_bin	加载数据后 GUI 热图出现重复斜线或通道布局异常。	该值必须等于 binary 文件总通道数，包括未用于 sorting 的非 ephys 通道；Neuropixels 数据常见 binary 通道数可大于实际 ephys 通道数。
batch_size	通道数较少，漂移估计不稳定或每批 spike 数不足。	默认 60000 samples，在 30000 Hz 下约 2 s。64 通道或更少的探针可适当增大 batch_size，以便每批包含更多 spikes 用于漂移估计。
nblocks	沿 probe 深度存在不同程度漂移，或低通道/稀疏探针漂移估计不可靠。	默认 nblocks=1 表示刚性配准；单 shank Neuropixels 可尝试 nblocks=5。约 64 通道以下或 contact 间距约 50 um 以上时，可设 nblocks=0 跳过漂移校正。
Th_universal / Th_learned	检测到的 spikes 偏少，或在 Phy 中观察到神经元随时间消失、重现。	二者分别控制 universal templates 与 learned templates 的检测阈值。若总体单位偏少可降低 Th_universal；若单位时间稳定性差可降低 Th_learned；建议每次降低 1-2。
tmin / tmax	记录开始或结束处存在明显 artifact。	仅将无明显污染的时间段纳入 sorting；tmax 可在 GUI 中使用 inf，在 API 中使用 np.inf 表示记录末尾。
nt	采样率不是 30 kHz，或需要重新匹配 spike waveform 时间窗。	nt 表示波形样本数及滤波对称 padding。默认值对应 30 kHz 下约 2 ms + 1 bin；40 kHz 时官方示例给出 nt=81 作为约 2 ms 的等效设置。
dmin / dminx	probe contact 间距不规则，或非 Neuropixels 几何导致模板空间布局不合适。	dmin 控制垂直模板中心间距，默认按 contact 中位距离估计；dminx 默认 32 um，适合 Neuropixels。其他探针可从横向 contact 中位距离开始设置。
min_template_size	contact 间距较宽，默认 universal template 空间包络过小。	默认 10 microns，表示生成 universal templates 时最小 Gaussian 空间包络的标准差；宽间距探针可考虑增大。
nearest_chans / nearest_templates	低通道或稀疏探针出现检测/数值不稳定。	nearest_chans 不能大于 probe 总通道数；少于 10 通道时需降低。约 64 通道以下且 contact 稀疏时，可将 nearest_templates 降至不超过通道数，以减少数值不稳定。

x_centers	probe contacts 呈 2D grid 排列，模板分组过密或计算效率下降。	多数情况无需手动设置。2D 阵列建议每 200-300 um 放置一个中心；例如宽 2000 um 的阵列可尝试 x_centers=10。
duplicate_spike_ms	Phy 中 otherwise good units 的 correlogram 在 0 ms 附近存在大峰值。	排序结束后，同一 unit 内该时间窗内的 spikes 会被视为 artifact 并移除；可适当增大，但官方警告不要超过 0.5 ms，以免影响 ACG/CCG 不应期估计。

总体上，Kilosort4 调参应优先处理数据结构性问题（n_chan_bin、dtype、fs、chanMap），随后处理检测灵敏度（Th_universal、Th_learned），再根据 probe 布局和漂移表现调整 nblocks、dmin、dminx、x_centers 等空间参数，最后再微调聚类资源参数（cluster_downsampling、max_cluster_subset）以在计算效率和排序精度间权衡。对于低通道、稀疏或非 Neuropixels-like 数据，应特别谨慎对待漂移校正和最近邻模板参数。下表列出了 Kilosort4 常见运行结果异常现象排查与关键参数修正建议表：

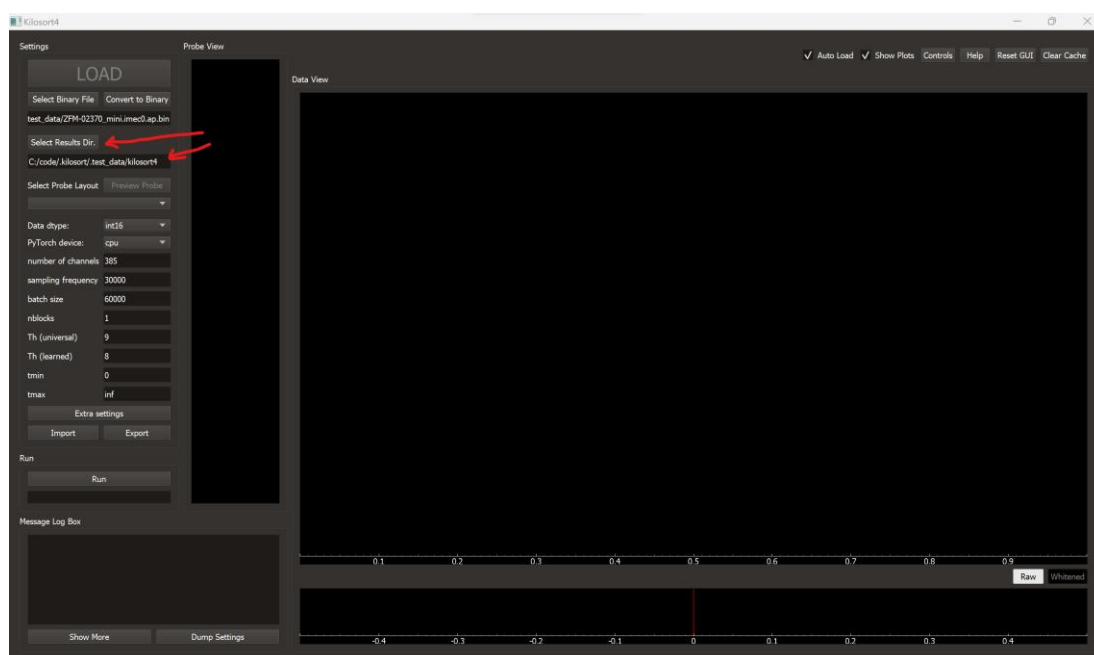
现象	优先检查/调整	官方建议的方向
GUI 热图出现重复斜线或通道错位	n_chan_bin、dtype、probe chanMap	n_chan_bin 必须等于 binary 总通道数。
检测 spike 很少	Th_universal、Th_learned	阈值每次降低 1 或 2；Th_universal 更影响总体单位数，Th_learned 与单位随时间消失/重现有关。
不同深度漂移程度不同	nblocks	增大 nblocks；单 shank Neuropixels 可尝试 5。
通道少或 contact 稀疏	nblocks 、 batch_size 、 nearest_templates	约 64 通道以下或间距约 50 um 以上时漂移估计可能不准，可设 nblocks=0；低通道可增大 batch_size。
采样率不是 30 kHz	nt、nt0min	nt 默认 61 对应 30 kHz 下约 2 ms + 1 bin；40 kHz 可参考 nt=81。
非 Neuropixels 或不规则间距	dmin 、 dminx 、 min_template_size	dmin 可手动设为垂直间距；dminx 可从横向 median distance 起步。
2D grid 探针模板过多	x_centers	按每 200-300 um 一个中心设置；例如宽 2000 um 可尝试 x_centers=10。
Phy 中 ACG 0 ms 附近峰异常	duplicate_spike_ms	可增大以移除重复 spike，但官方警告不要超过 0.5 ms。



GUI 重置与运行状态界面示例

8 输出文件解析

Kilosort4 默认将结果保存到 binary 数据同目录下的 kilosort4 子目录中，文件兼容 Phy。需要特别注意的是：这里的 `template` 不再指检测阶段的 `universal` 或 `learned templates`，而是指每个 `cluster` 的平均 `spike waveform`；在 Kilosort4 代码语境中更接近 `clusters`，但为兼容 Phy 仍称为 `template`。



GUI 中的结果目录设置

文件	数据结构	含义
amplitudes.npy	(n_spikes,)	每个 spike 的 amplitude，按 PC features 的 L2 norm 计算。
channel_map.npy	(n_channels,)	等同 probe['chanMap']，映射 binary 行到 probe contact。
channel_positions.npy	(n_channels, 2)	probe['xc'] 与 probe['yc'] 合并，单位微米。
channel_shanks.npy	(n_channels,)	每个通道对应的 shank index。
cluster_Amplitude.tsv	(n_templates,)	每个 template 的 amplitude，按 template 的 L2 norm 计算。
cluster_ContamPct.tsv	(n_templates,)	每个 template 的污染率，基于不应期违例相对泊松期望估计。
cluster_KSLabel.tsv	(n_templates,)	good 或 mua 标签。
cluster_group.tsv	(n_templates,)	与 cluster_KSLabel.tsv 相同。
kept_spikes.npy	(n_spikes,)	布尔 mask；重复 spike 移除后保留为 True。
ops.npy	N/A	保存 sorting 过程中状态变量的字典，官方建议用 kilosort.io.load_ops 读取。
params.py	N/A	Phy 使用的设置，如数据位置和采样率。
pc_features.npy	(n_spikes, n_pcs, nearest_chans)	每个 spike 在其 template 最近通道上的时间特征。
pc_feature_ind.npy	(n_templates, nearest_chans)	每个 template 最近通道的索引。
similar_templates.npy	(n_templates, n_templates)	模板间相关性相似度。
spike_clusters.npy	(n_spikes,)	每个 spike 分配到的 template/cluster 整数编号。
spike_templates.npy	(n_spikes, 2)	官方页面说明与 spike_clusters.npy 相同。
spike_positions.npy	(n_spikes, 2)	每个 spike 相对 probe 布局的估计 x/y 位置，单位微米。
spike_times.npy	(n_spikes,)	每个 spike 波形峰值的 sample index。
templates.npy	(n_templates, nt, n_channels)	完整 time x channels 的 template shape。
templates_ind.npy	(n_templates, n_channels)	cluster 定义通道索引；KS4 中总是全部通道，但 Phy 要求该文件。

whitening_mat.npy	(n_channels, n_channels)	应用于数据 whitening 的矩阵。
whitening_mat_inv.npy	(n_channels, n_channels)	whitening matrix 的逆矩阵。
whitening_mat_dat.npy	(n_channels, n_channels)	当前与 whitening_mat.npy 相同，用于备份原始 whitening 版本。

结果校正一般从 params.py 启动 Phy：在 Kilosort4 的结果目录中执行 phy template-gui params.py。Kilosort4 会根据估计污染率 <20% 的不应期标准自动设置 Phy 中的 good units，但人工修正仍是质量控制的重要步骤。

```
cd <DATA_DIRECTORY>/kilosort4
phy template-gui params.py
```

9 参考资料

1) Kilosort4 官方文档首页: <https://kilosort.readthedocs.io/en/latest/index.html>

2) Kilosort4 Github: <https://github.com/MouseLand/Kilosort>

3) 官方参数源码: <https://github.com/MouseLand/Kilosort/blob/main/kilosort/parameters.py>

4) SpikeInterface 数据加载/转换教程:
https://kilosort.readthedocs.io/en/latest/tutorials/load_data.html

5) SpikeInterface 官方文档: <https://spikeinterface.readthedocs.io/en/stable/overview.html>

6) Pachitariu M, Sridhar S, Pennington J, Stringer C. Spike sorting with Kilosort4. Nat Methods. 2024 May;21(5):914-921. doi: 10.1038/s41592-024-02232-7.